Web Service 개발 Mini-Project

작성자: 한상윤(PM), 김주환, 방다원, 지다은

> 1. 프로젝트 개요 및 핵심 가치 정의
> 2. PainPoints & Solutions
> 3. 요구사항 분석
> (MVP & Future Extensions)
> 4. 시스템 설계 개요 및 아키텍처
> 5. 데모 스냅샷
> 6. 요약 및 결론

Enterprise AI Agent Management Platform (A!rport)

본 서비스는 조직 내 AI 에이전트를 개인 단위의 실험적 도구가 아닌, **전사적으로 관리·평가·통제되는 핵심 운영 자산으로 전환하는 AI 운영 플랫폼(AIOps 기반 Agent OS)**이다.

현재 기업은 빠르게 AI를 도입하고 있지만, 이는 개별 생산성 향상 수준에 머무르고 있으며 조직 전체의 전략적 자산으로 연결되지 못하고 있다. 본 서비스는 이러한 한계를 극복하고, 에이전트를 중심으로 한 **AI 자산화, 성능 표준화, 비용 통제, 협업 자동화 구조를 동시에 구축**한다.

이를 통해 기업은 분산된 AI를 통합 관리하여 자산 가시성을 확보하고, 성능 비교를 통해 우수 에이전트를 중심으로 품질을 표준화하며, 예산 기반 운영으로 AI 비용을 통제하고, MoA 기반 협업을 통해 복잡한 업무까지 자동화할 수 있다. 또한 개인의 AI 활용을 KPI와 연결함으로써 조직 전체의 AI 활용 역량을 정량적으로 강화할 수 있다.

결과적으로 본 서비스는 AI를 단순히 사용하는 조직을 넘어, **AI를 기반으로 의사결정과 업무가 이루어지는 조직으로의 구조적 전환**을 가능하게 한다.

핵심 가치 (Why this service matters)

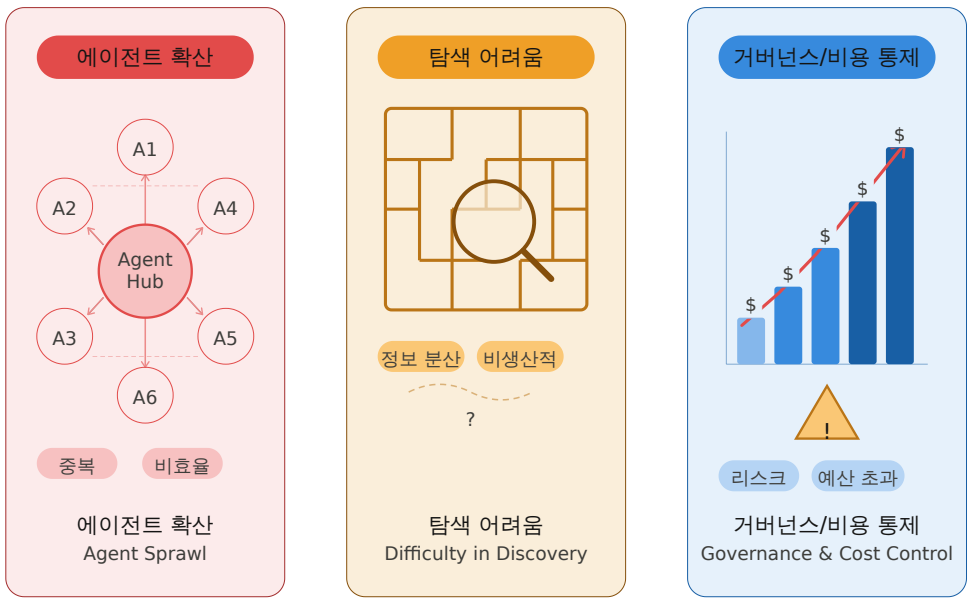
본 서비스는 AI를 “잘 쓰는 도구”에서 “조직 운영 시스템”으로 전환시키는 것을 핵심 가치로 한다.

- 분산된 AI → 전사적 자산으로 통합
- 감각적 선택 → 데이터 기반 의사결정
- 무제한 사용 → 예산 기반 통제
- 단일 도구 → 협업 시스템(MoA)

이를 통해 기업은 단순한 생산성 향상을 넘어,
AI 기반 의사결정 체계와 자동화된 업무 프로세스를 갖춘 조직으로 진화하게 된다.

---

1. Pain Points



1. 에이전트 관리 부재 및 중복 개발로 인한 비효율

조직 내 다양한 부서에서 AI 에이전트가 개별적으로 생성되고 있으나, 이를 통합적으로 관리하는 체계가 부재하여 현재 어떤 에이전트가 존재하는지 파악하기 어렵다. 개인 단위로 생성된 에이전트는 조직 자산으로 축적되지 못하며, 유사한 기능을 수행하는 에이전트가 반복적으로 생성되고 있다. 기존 에이전트에 대한 탐색 및 재사용 구조가 마련되어 있지 않아 불필요한 개발 및 운영 비용이 지속적으로 증가하고 있다.

2. 성능 검증 불가 및 복합 업무 활용의 한계

에이전트 간 성능을 비교할 수 있는 정량적 기준이 없어 어떤 에이전트가 더 우수한지 판단하기 어렵고, 품질이 검증되지 않은 에이전트가 업무에 활용되면서 신뢰도 문제가 발생하고 있다. 또한 대부분의 에이전트가 단일 기능 중심으로 설계되어 있어 분석·판단·생성이 결합된 복합 업무의 자동화가 어려우며, AI가 여전히 보조 도구 수준에 머무르고 있다.

3. 비용 통제 체계 부재

AI 사용량이 증가함에도 불구하고 부서별·개인별 사용량 추적 및 비용 관리 체계가 마련되어 있지 않아, 비용이 불투명하게 누적되고 자원 사용 효율성이 저하되고 있다.

---

2. Solutions

> 본 서비스는 조직 내 AI 활용 과정에서 발생하는 세 가지 핵심 문제를 해결하기 위해 다음과 같은 기능 구조를 제공한다.

1. 에이전트 확산 → Agent Catalog

에이전트 증가로 인한 현황 파악 어려움과 중복 개발·활용 분산 문제를 해결하기 위해 카탈로그 등록 및 분류 관리 기능을 제공한다. 이를 통해 에이전트 중복 개발을 감소시키고, 기존 에이전트의 재사용률을 높이며, 전사 AI 자산 가시성을 확보한다.

2. 탐색/사용 어려움 → Search & Compare

적합한 에이전트 탐색이 어렵고 성능 비교 및 목적 확인이 불가한 문제를 해결하기 위해 에이전트 검색 및 비교 기능을 제공한다. 이를 통해 에이전트 탐색 시간을 단축하고, 업무에 적합한 에이전트 선택을 가능하게 하며, 조직 단위 활용을 확대한다.

3. 거버넌스/비용 통제 → Access & Budget Control

사용 권한 및 비용 관리 부재, AI 사용량 추적 어려움을 해결하기 위해 권한 승인 및 예산 관리 기능을 제공한다. 이를 통해 부서별 AI 비용 추적을 가능하게 하고, 사용 권한 통제 체계를 확립하며, AI 자원 사용을 최적화한다.

본 서비스는 에이전트를 카탈로그화하여 조직 자산으로 관리하고, 성능을 데이터 기반으로 비교하며, 비용을 예산 기반으로 통제하는 통합 구조를 제공한다. 이를 통해 AI 활용 방식을 개별 단위의 도구 사용에서 조직 차원의 운영 체계로 전환한다.

---

3. 요구사항 분석 — MVP 핵심 서비스

서비스	설명	기대 변화
에이전트 카탈로그	사내 AI 에이전트 목록·상세 조회, 카테고리별 탐색, 활용도 기반 의사결정 지원	조직 내 AI 자산 가시성 확보
에이전트 등록 및 구독	사원·부서 단위 에이전트 등록, 사용 신청 및 승인·반려 기반 권한 관리	1인 N 에이전트 생성 구조 지원, 무분별한 AI 사용 방지
에이전트 사용 비용 관리	부서별 예산 설정, 사용량 기반 실시간 비용 차감, 예산 초과 시 사용 제한	예산 범위 내 AI 사용 보장, 비용 투명성 확보
에이전트 실행 연동	카탈로그에서 에이전트 직접 실행, 분석 결과 생성 및 제공	탐색에서 실행까지 끊김 없는 사용자 경험 제공
챗봇	업무 유형 기반 에이전트 자동 추천, 단순·복합 질의에 따른 모델 분기	업무 맥락에 맞는 에이전트 즉시 연결
AI 활용 성과 관리	사원별 에이전트 생성 수 집계, AI 활용도 측정, 조직 단위 현황 분석	조직 AI 생산성 측정 및 KPI 연계 기반 마련
(확장) 에이전트 성능 비교	성능 비교 대시보드, 응답 품질 및 활용도 비교	최적 에이전트 선택 지원
(확장) MOA	업무별 에이전트 조합 구성, 단계별 순차 실행	복합 업무 자동화
(확장) AI 성과 관리 고도화	조직 AI 생산성 분석 및 KPI 연계 심화	고과 연계 체계 완성

I. 에이전트 카탈로그

사내에 존재하는 모든 AI 에이전트를 한 곳에서 탐색하고 현황을 파악할 수 있는 중앙 카탈로그 서비스다. 조직 내 AI 자산의 통합 가시성 확보를 목표로 하며, 에이전트 목록 및 상세 정보 조회, 카테고리·유형별 탐색, 사용 횟수 기반 인기 에이전트 확인, 활용도 기반 의사결정 지원 기능을 제공한다.

II. 에이전트 등록 및 구독

사원이 직접 에이전트를 등록하고 부서 단위로 구독 신청하는 온보딩 흐름을 일원화한 서비스다. 1인 N 에이전트 생성 구조를 지원하는 동시에 무분별한 AI 사용을 방지하는 것을 목표로 한다. 사원·부서 단위 에이전트 등록, 카테고리 및 활용 목적 정의, 부서 기준 사용 신청, 승인·반려 기반 권한 관리 기능을 포함한다.

III. 에이전트 사용 비용 관리

부서별 AI 사용량을 기반으로 비용을 통제하는 서비스다. 예산 범위 내에서 AI 사용을 보장하는 것을 목표로 하며, 부서별 예산 설정 및 관리, 사용량 기반 실시간 비용 차감, 사용 이력 추적, 예산 초과 시 사용 제한 기능을 제공한다.

IV. 에이전트 실행 연동

카탈로그에서 탐색한 에이전트를 실제로 실행하기까지의 흐름을 연결하는 서비스다. 탐색에서 실행까지 끊김 없는 사용자 경험 제공을 목표로 하며, 업무 에이전트 직접 실행, 분석 결과 생성 및 제공, 추천 기반 에이전트 활용 기능을 지원한다.

V. 챗봇

업무 난이도에 따라 최적의 에이전트를 자동으로 매칭하는 서비스다. 업무 맥락에 맞는 에이전트를 즉시 연결하는 것을 목표로 하며, 업무 유형 기반 에이전트 자동 추천, 유사 에이전트 탐색, 구독 에이전트 내 자동 배정, 단순·복합 질의에 따른 모델 분기 기능을 제공한다.

VI. AI 활용 성과 관리

사원별 에이전트 생성 및 활용 성과를 추적하는 서비스다. 조직 AI 생산성 측정 및 고과 연계 기반 마련을 목표로 하며, 사원별 에이전트 생성 수 집계, AI 활용도 측정, 조직 단위 현황 분석, KPI 연계 확장 구조 지원 기능을 포함한다.

Partial & Future Extension

> 현재 일부 구현 또는 향후 확장이 예정된 서비스로, 아래 3개 항목을 정의하였다.

I. 에이전트 성능 비교

에이전트 간 성능을 비교하고 분석하는 서비스다. 최적 에이전트 선택을 지원하는 것을 목표로 하며, 성능 비교 대시보드 제공, 응답 품질 및 활용도 비교, 향후 실데이터 기반 분석 고도화를 포함한다.

II. MOA (Multi Agent 협업)

여러 에이전트가 협업하여 복합 업무를 자동화하는 서비스다. 복합 업무의 단계별 자동 처리를 목표로 하며, 업무별 에이전트 조합 구성, 단계별 에이전트 순차 실행, 향후 표준 협업 구조 확장을 포함한다.

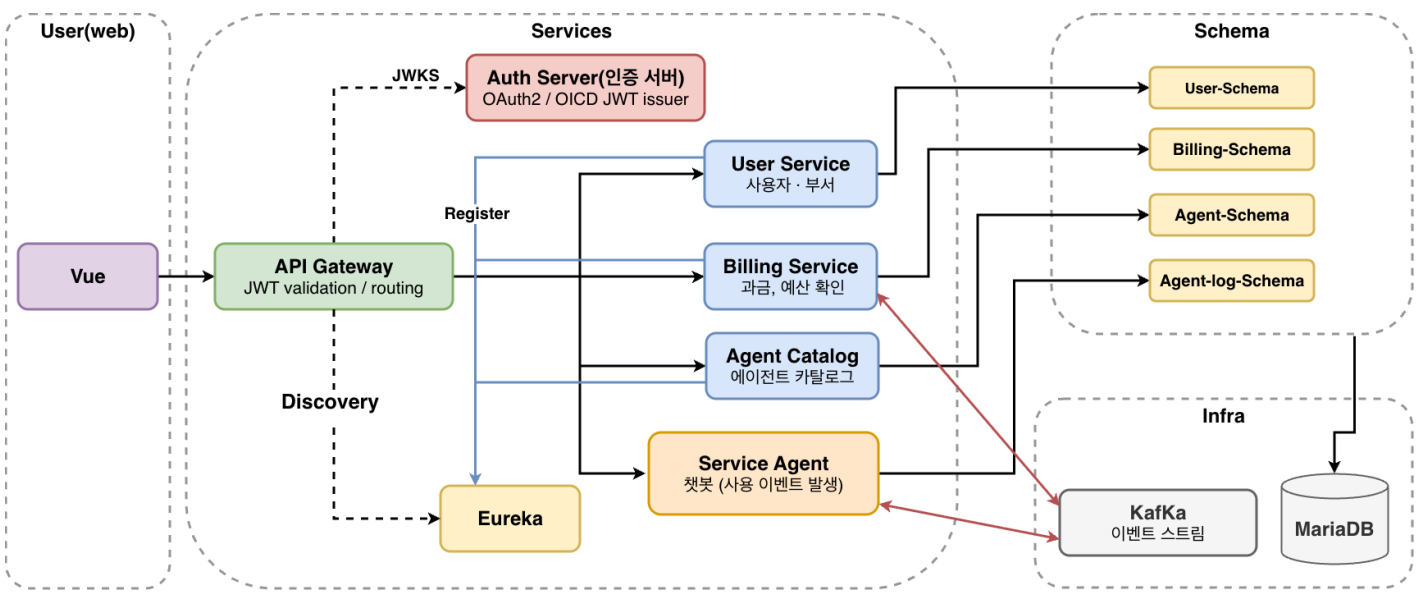
III. AI 활용 성과 관리 고도화

MVP에서 제공하는 기본 성과 추적을 바탕으로 조직 AI 생산성 분석 및 KPI 연계를 고도화하는 단계다. 사원별 에이전트 생성 수 추적, AI 활용도 측정, KPI 연계 확장 기능을 점진적으로 확대할 예정이다.

---

4. 시스템 설계 개요

본 시스템은 Vue 기반 프론트엔드에서 시작하여 API Gateway를 통해 단일 진입점을 구성하고, Gateway는 Eureka 기반 서비스 디스커버리를 활용하여 각 마이크로서비스로 요청을 라우팅하는 구조로 설계되었다.



백엔드는 User Service, Agent Catalog, Billing Service로 분리하였으며, 각 서비스는 독립적으로 배포되는 MSA 구조를 따른다. 클라우드 네이티브 설계 원칙에 따라 서비스별 폴더 구조와 독립 빌드 환경을 구성하였으며,

데이터베이스 설계: 공유 인스턴스 내 논리적 격리

공용 MariaDB를 사용하되 각 서비스별로 **독립된 스키마**를 부여하여 관리하도록 하였다. 이를 통해 물리적 인프라 비용은 절감하면서도, 서비스 간 데이터 침범을 차단하여 **데이터 권한 분립과 무결성**을 확보할 수 있다. 각 서비스는 오직 자신에게 할당된 스키마에만 접근하며, 타 서비스의 데이터가 필요할 경우 API 또는 이벤트를 통해서만 소통한다.

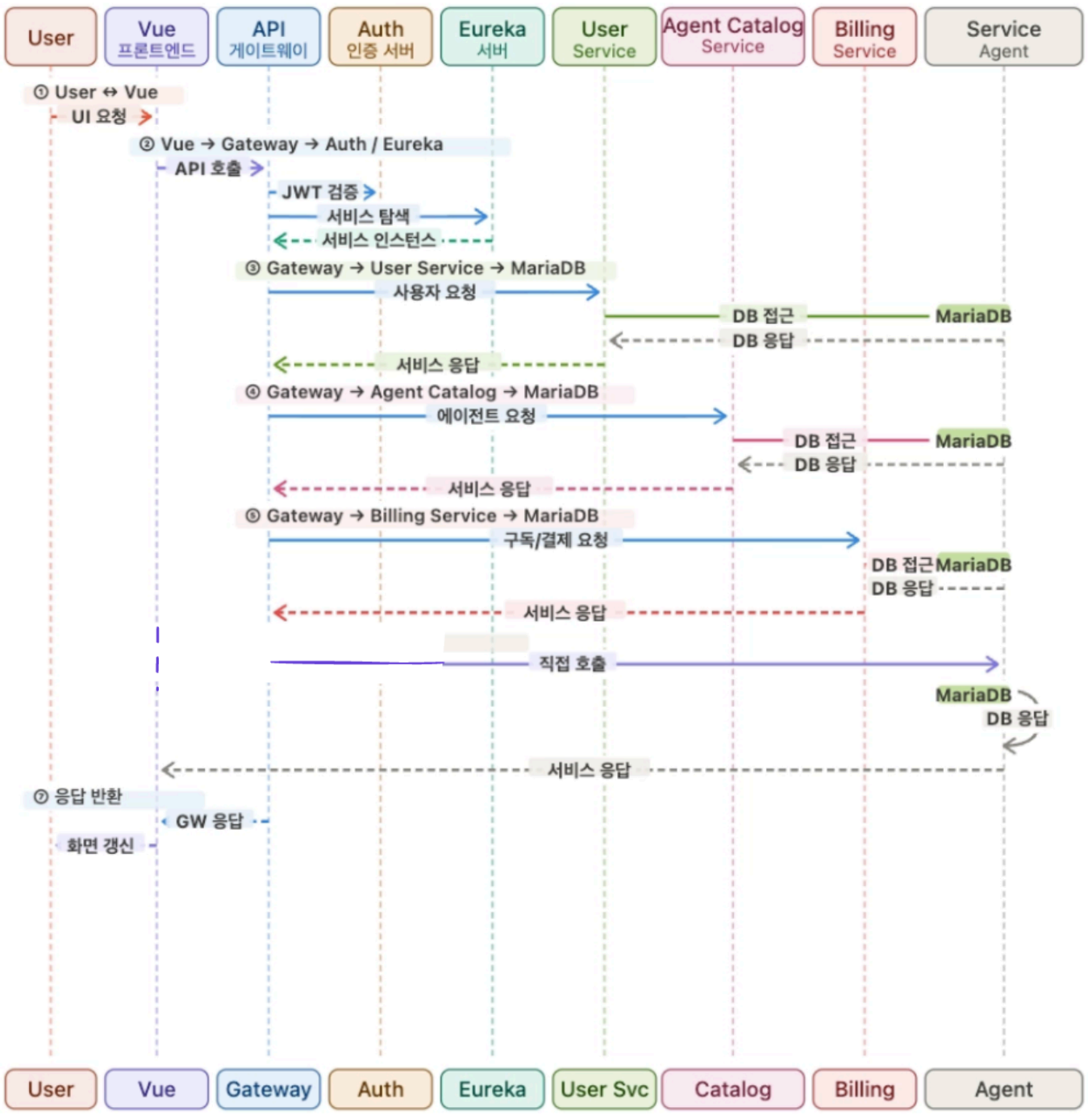
- 에이전트 호출 시 발생하는 사용 이벤트를 비동기 스트림으로 처리하여 Billing Service와 연동함으로써 사용량 기반 비용 차감의 안정성을 확보하였다. 분산 환경에서의 데이터 일관성을 위해 별도의 중앙 제어자가 없는 **Choreography 기반 Saga 패턴**을 적용했다.
- 인증은 OAuth2/OIDC 기반 Auth Server와 JWT 기반 리소스 서버 구조를 통해 처리되어 보안성과 확장성을 동시에 확보한다.
- AI 처리 영역은 FastAPI로 분리하여 LLM 및 에이전트 실행을 독립적으로 수행할 수 있도록 하였으며, Kafka를 통해 Billing Service와 연동하여 AI 사용량도 동일한 비용 처리 흐름에 포함되도록 하였다.

개발 방법론으로는 애자일 방식을 부분적으로 적용하였다. 요구사항 정의 및 설계 단계에서는 애자일 방식으로 유연하게 반복하고, 구현 단계에서는 확정된 MVP 범위를 기준으로 진행하는 하이브리드 방식을 채택하였다.

---

5. 아키텍처 설계

- 본 시스템은 Vue 기반 사용자 인터페이스를 시작으로 API Gateway를 거쳐 각 마이크로서비스로 요청이 전달되며, 데이터 저장과 이벤트 처리는 각각 MariaDB와 Kafka를 통해 이루어진다. 실제 AI 추론은 별도의 FastAPI 기반 Agent 서버에서 수행된다.
- 사용자는 프론트엔드에서 에이전트를 탐색하거나 실행 요청을 보내며, 모든 요청은 Gateway를 통해 인증과 라우팅이 처리되고, 이후 각 서비스로 분산되어 처리된다.
    - 서비스 간 연결은 고정된 주소가 아닌 서비스 이름 기반으로 이루어지며, 이를 위해 Eureka를 활용하여 동적으로 서비스 위치를 관리한다.



5.1 시스템 구조

- 본 시스템은 Vue 기반 사용자 인터페이스를 시작으로 API Gateway를 거쳐 각 마이크로서비스로 요청이 전달되며, 데이터 저장과 이벤트 처리는 각각 MariaDB와 Kafka를 통해 이루어진다. 실제 AI 추론은 별도의 FastAPI 기반 Agent 서버에서 수행된다.
- 사용자는 프론트엔드에서 에이전트를 탐색하거나 실행 요청을 보내며, 모든 요청은 Gateway를 통해 인증과 라우팅이 처리되고, 이후 각 서비스로 분산되어 처리된다.
- 서비스 간 연결은 고정된 주소가 아닌 서비스 이름 기반으로 이루어지며, 이를 위해 Eureka를 활용하여 동적으로 서비스 위치를 관리한다.

5.2 계층별 역할 및 구성

- **사용자 인터페이스 계층 (Vue)**
  사용자 인터페이스 계층은 Vue 기반으로 구성되며, 사용자의 요청을 API 호출 형태로 변환하고 시스템과의 상호작용을 담당한다. 단순한 화면 제공을 넘어, 에이전트 탐색, 실행 요청, 결과 확인까지 전체 사용자 흐름을 통합적으로 관리하며, 사용자 의도를 시스템이 처리 가능한 형태로 변환하는 역할을 수행한다.

- **API Gateway 계층 (Spring Cloud Gateway)**
  API Gateway는 시스템의 중앙 제어 계층으로, 모든 외부 요청의 단일 진입점 역할을 수행한다. 요청에 대해 JWT 기반 인증 검증을 수행하고, 서비스 라우팅 및 공통 정책을 적용한다. 이를 통해 내부 마이크로서비스 구조를 외부로부터 완전히 분리하고, 시스템의 확장성과 보안성을 동시에 확보하도록 설계하였다.

- **인증 계층 (OAuth2 / OIDC)**
  인증 계층은 OAuth2 및 OIDC 기반으로 구성되며, Auth Server에서 토큰을 발급하고 API Gateway에서 이를 검증하는 구조를 가진다. 각 마이크로서비스는 인증 로직을 직접 처리하지 않고, 인증된 사용자 정보를 전달받아 비즈니스 로직에만 집중하도록 설계하였다.

- **서비스 디스커버리 계층 (Eureka)**
  서비스 디스커버리 계층은 “서비스들의 위치를 관리해주는 주소록” 역할을 한다. 서비스 디스커버리 계층은 Eureka를 활용하여 구현되었으며, 각 서비스는 실행 시 자신의 정보를 등록하고 Gateway는 서비스 이름 기반으로 요청을 전달한다. 이를 통해 서비스의 IP나 포트를 코드에 고정하지 않고도 동적으로 연결이 가능하며, 서비스 확장 및 장애 대응에 유연한 구조를 제공한다.

- **마이크로서비스 계층 (Domain Services)**
  마이크로서비스 계층은 기능별로 명확히 분리된 구조를 가지며, 각 서비스는 독립적으로 배포 및 확장이 가능하도록 설계되었다.
  User Service는 사용자 및 조직 정보를 관리하고, Agent Catalog Service는 에이전트의 메타데이터를 관리하며, Billing Service는 사용량 및 비용 처리를 담당한다. 또한 FastAPI 기반의 AI Agent는 실제 LLM 추론 및 실행을 담당하는 핵심 실행 계층으로, 에이전트 요청을 받아 결과를 생성하는 역할을 수행한다.

- **데이터 계층 (MariaDB)**
  데이터 계층에서는 MariaDB를 사용하여 사용자, 에이전트, 결제 및 사용 이력과 같은 정형 데이터를 저장한다. 트랜잭션 기반의 데이터 처리 구조를 통해 데이터 정합성을 유지하며, 각 서비스는 필요에 따라 해당 데이터를 조회 및 갱신하도록 설계되어 있다.

- **이벤트 처리 계층 (Kafka 기반 비동기 구조)**
  이벤트 처리 계층은 "서비스 간 신호를 전달하는 메시지 시스템" 역할을 한다. 이벤트 처리 계층은 Kafka를 기반으로 구성되며, 서비스 간 결합도를 낮추고 비동기 처리를 가능하게 한다. 특정 서비스에서 이벤트가 발생하면 Kafka에 메시지를 발행하고, 이를 구독하고 있는 다른 서비스가 해당 이벤트를 감지하여 자동으로 트리거되는 구조를 가진다. 예를 들어 Billing Service에서 결제 이벤트가 발생하면 Kafka를 통해 전달되고, Service Agent가 이를 수신하여 후속 처리나 분석 작업을 수행한다. 이와 같은 이벤트 기반 아키텍처를 통해 시스템의 확장성과 안정성을 동시에 확보하였다.

  핵심 기술 구성에 대한 이해

  - **Eureka는** 서비스들의 위치 정보를 관리하는 시스템으로, 각 서비스가 실행되면 자신의 정보를 등록하고 Gateway는 서비스 이름만으로 해당 서비스에 요청을 전달할 수 있도록 한다. 이를 통해 서비스 확장이나 변경 시에도 코드 수정 없이 유연하게 대응할 수 있다.
  - **Kafka는** 서비스 간 데이터를 비동기적으로 전달하기 위한 이벤트 스트리밍 시스템으로, 실행 요청과 별개로 발생하는 로그나 과금 데이터를 처리하는 데 사용된다. 이를 통해 사용자 요청 처리 속도를 유지하면서도 데이터 처리를 안정적으로 수행할 수 있다.

5.3 요청 처리 흐름

- 사용자가 Vue UI에서 에이전트 실행 요청을 생성하면 해당 요청은 Gateway로 전달되고, Gateway는 JWT 토큰을 검증한 뒤 Eureka를 통해 대상 서비스 위치를 확인한다.
- 이후 요청은 필요에 따라 User Service, Catalog Service, Billing Service를 거쳐 실행 가능 여부를 판단하고, 실제 AI 실행은 FastAPI 기반 Agent 서버에서 수행된다.
- 실행 결과는 즉시 사용자에게 반환되며, 동시에 사용 로그는 Kafka를 통해 전달되어 Billing Service에서 비용 처리가 이루어진다. 이 과정에서 실행과 비용 처리를 분리하여 시스템 응답 속도와 안정성을 동시에 확보하였다.

5.4 구현 과정에서의 주요 난이도와 해결 방식

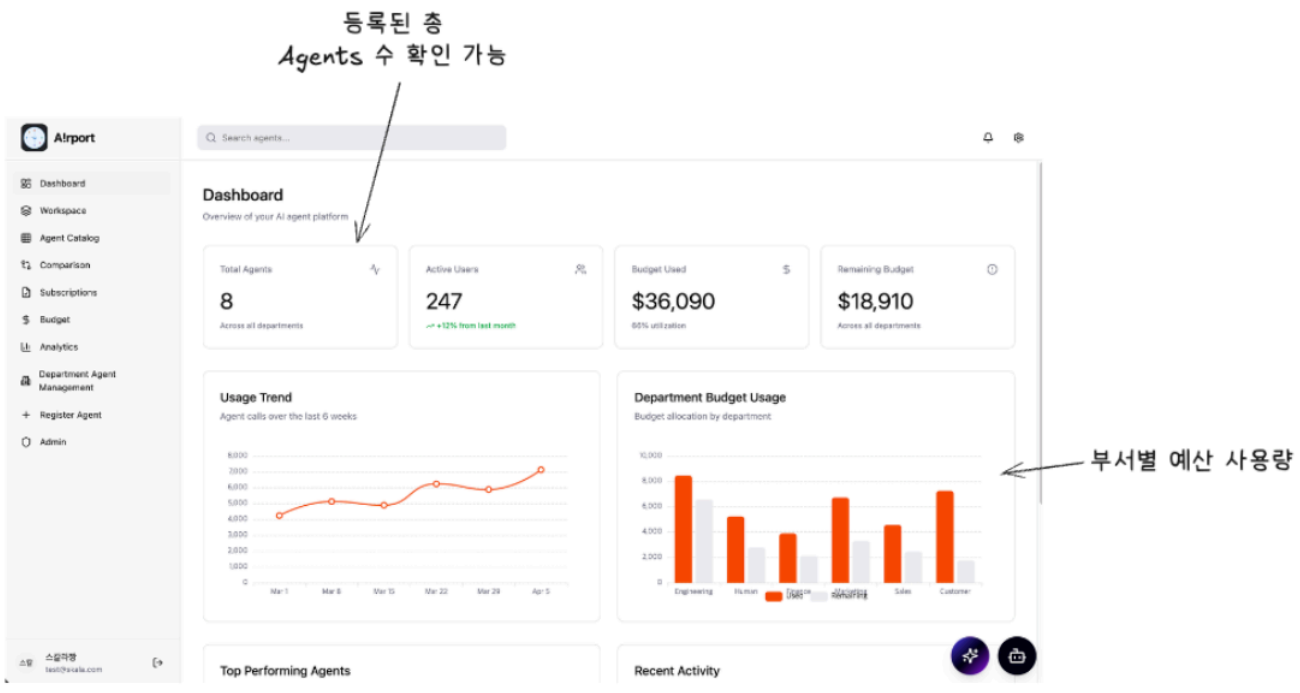
- Gateway와 인증 서버를 연동하는 과정은 단순 로그인 구현을 넘어 모든 요청 흐름에서 인증을 유지해야 하기 때문에 난이도가 높은 영역이었다. 이를 해결하기 위해 인증 처리는 Gateway에서 수행하고 각 서비스에서는 인증 로직을 제거하여 역할을 명확히 분리하였다.
- MSA 구조에서 가장 중요한 문제는 데이터 분할이었으며, 잘못 설계할 경우 서비스 간 결합도가 높아지고 트랜잭션 문제가 발생할 수 있다. 본 시스템에서는 초기 단계에서 단일 MariaDB를 사용하되 서비스별로 논리적 도메인을 분리하여 관리함으로써 안정성과 확장성을 동시에 확보하였다.
- AI 모델 실행은 일반 서비스와 다른 실행 환경과 높은 리소스를 요구하기 때문에 FastAPI 기반으로 별도 분리하여 구성하였으며, 이를 통해 AI 실행을 독립적으로 확장할 수 있도록 설계하였다.
- 비용 처리 및 로그 기록을 실시간으로 처리할 경우 시스템 부하가 증가하기 때문에 Kafka를 활용하여 비동기 처리 구조를 도입하였으며, 이를 통해 사용자 응답 속도를 유지하면서도 안정적인 데이터 처리가 가능하도록 하였다.

5.5 특징

- REST API 기반으로 모든 기능을 구성하여 각 서비스가 독립적인 에이전트처럼 동작하도록 설계하였다.
- 클라우드 네이티브 구조를 기반으로 서비스 간 결합도를 낮추고 확장성을 확보하였다.
- AI 실행과 관리 기능을 분리하여 성능과 안정성을 동시에 고려하였다.
- 데이터 흐름을 동기 처리와 비동기 처리로 분리하여 시스템 효율을 극대화하였다.
- 운영 데이터를 기반으로 향후 모델 성능 개선 및 자동화로 확장 가능한 구조를 고려하였다.

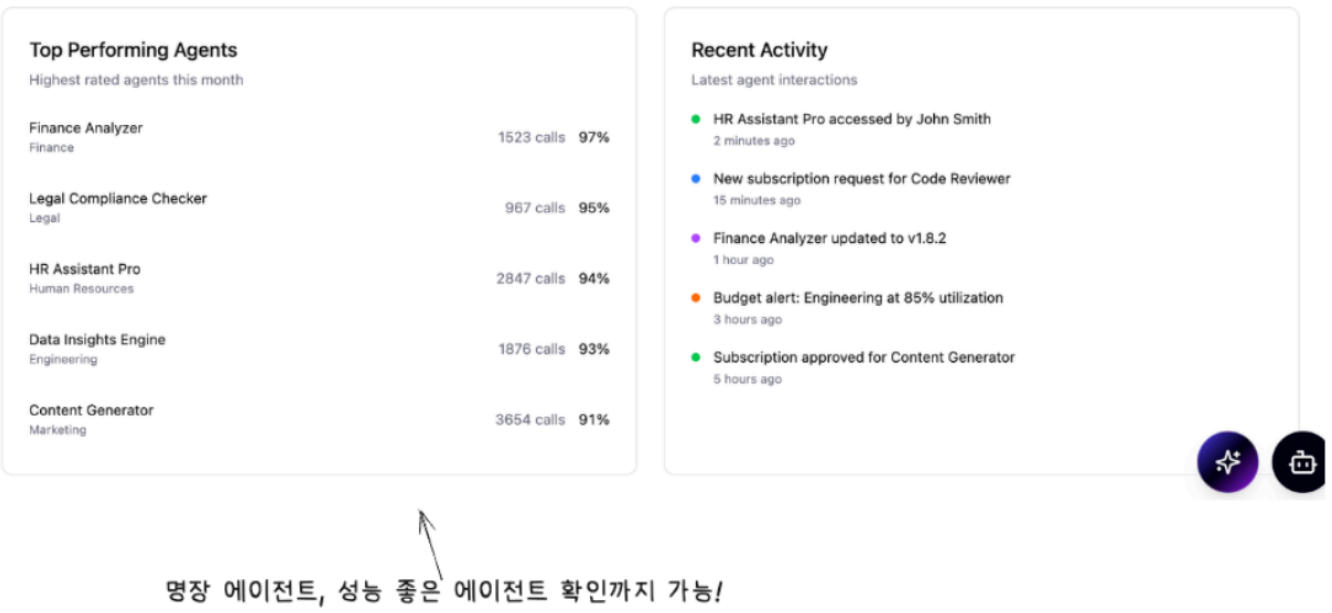
DEMO

1. Dashboard



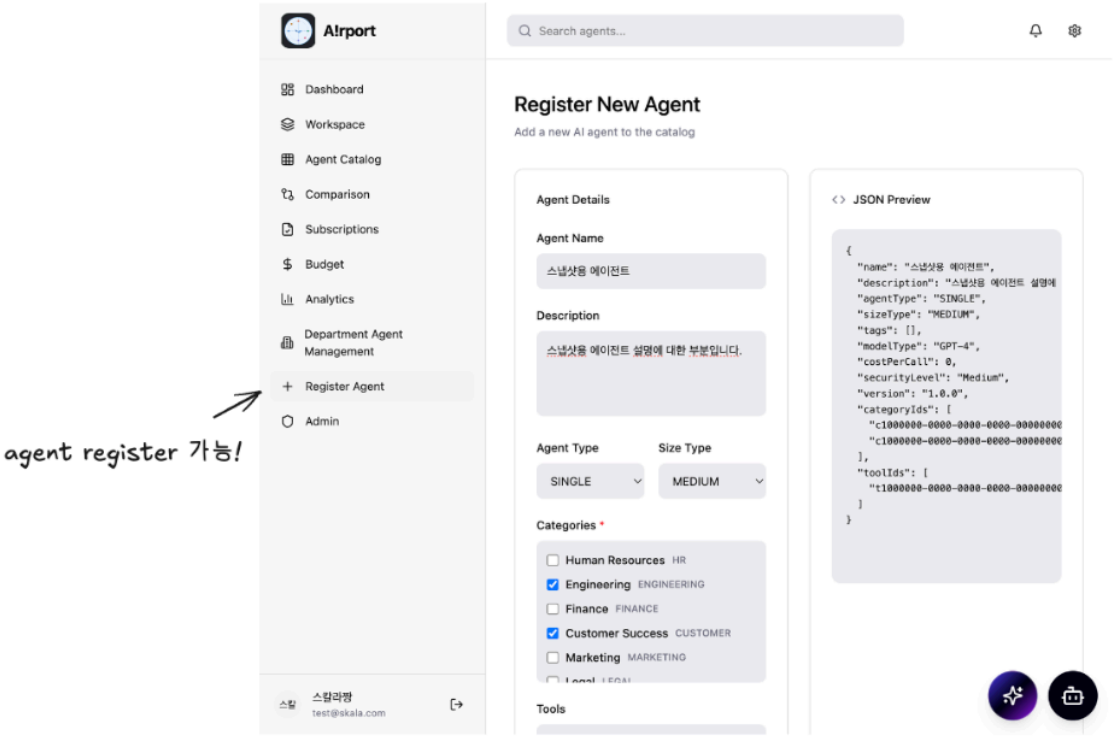
> 전사 AI 운영 현황을 한눈에 파악할 수 있는 메인 대시보드다. 등록된 전체 에이전트 수, 활성 사용자 수, 예산 사용량 및 잔여 예산을 상단 지표로 제공하며, 최근 6주간의 에이전트 호출 추이와 부서별 예산 사용 현황을 차트로 시각화하였다. 조직 단위의 AI 활용 현황을 실시간으로 모니터링할 수 있도록 설계하였다.

2. Top Performing Agents / Recent Activity



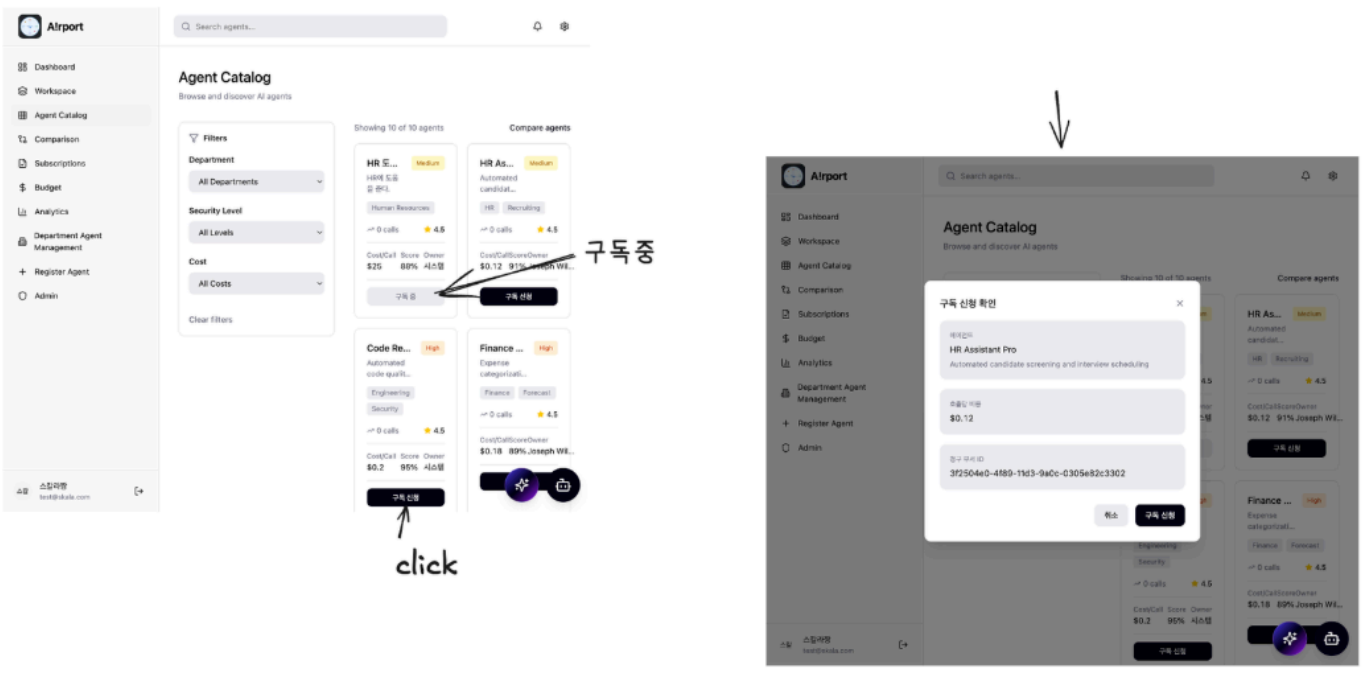
> 사용 횟수와 만족도 기반으로 상위 에이전트를 순위화하여 제공하며, 조직 내에서 검증된 에이전트를 식별하고 재사용을 촉진하는 역할을 한다. Recent Activity 영역에서는 구독 신청, 예산 경고, 에이전트 업데이트 등 최근 발생한 주요 이벤트를 실시간으로 확인할 수 있다.

3. Register New Agent



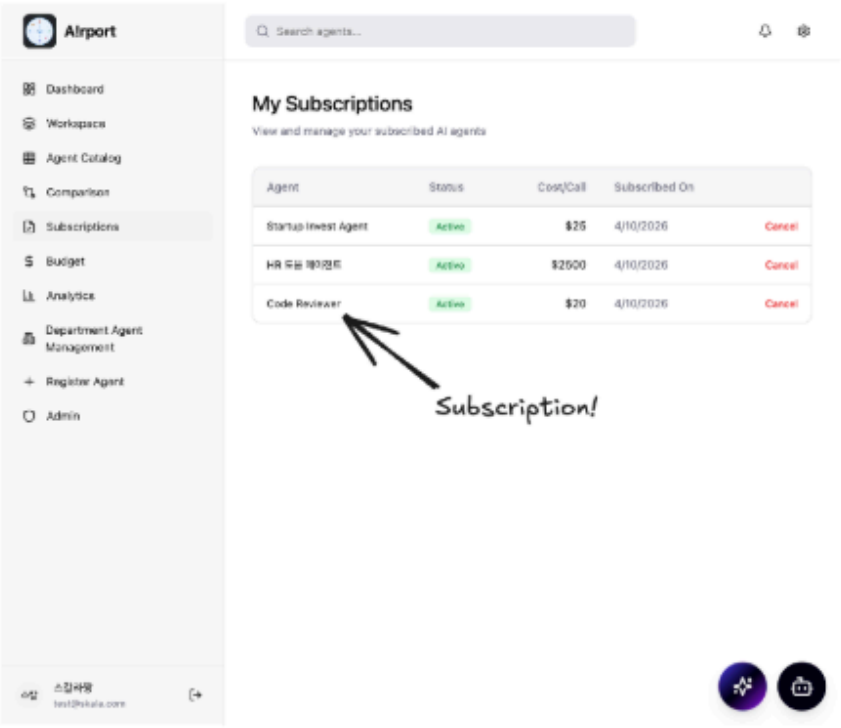
> 사원이 직접 에이전트를 등록하는 화면이다. 에이전트 이름, 설명, 유형, 카테고리, 연동 도구 등을 입력하면 우측에 JSON 형태로 즉시 미리보기가 제공된다. 개인이 개발한 에이전트를 조직 자산으로 등록하고 공유하는 1인 N 에이전트 구조의 핵심 진입점이다.

4. Agent Catalog



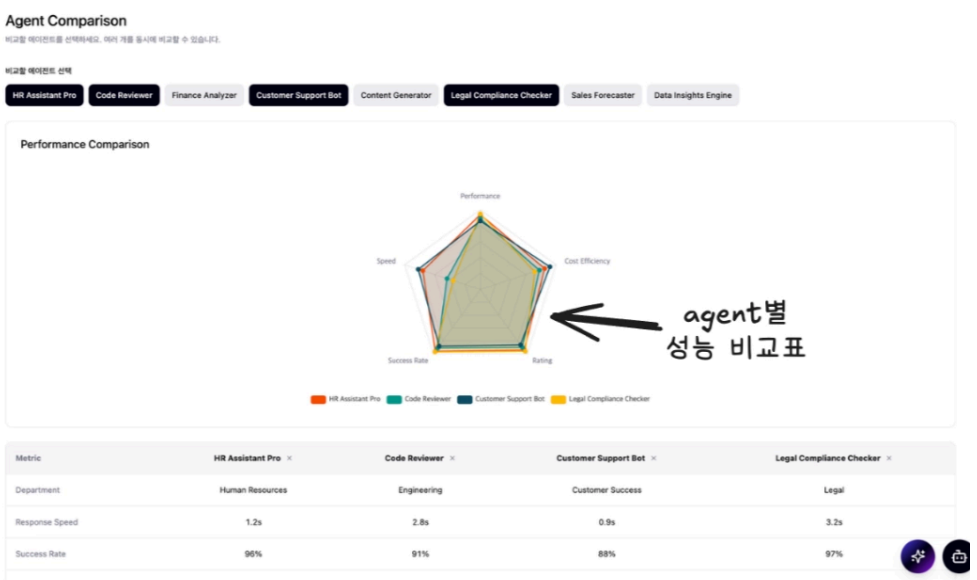
> 사내에 등록된 전체 에이전트를 탐색하고 구독을 신청하는 화면이다. 부서, 보안 등급, 비용 기준으로 필터링이 가능하며, 각 에이전트 카드에는 카테고리, 호출 수, 평점, 비용, 소유자 정보가 함께 표시된다. 구독 신청 버튼을 통해 부서 ID 기반의 승인 요청이 즉시 발송되며, 승인 전까지는 사용이 제한되는 권한 관리 구조를 구현하였다.

5. My Subscriptions



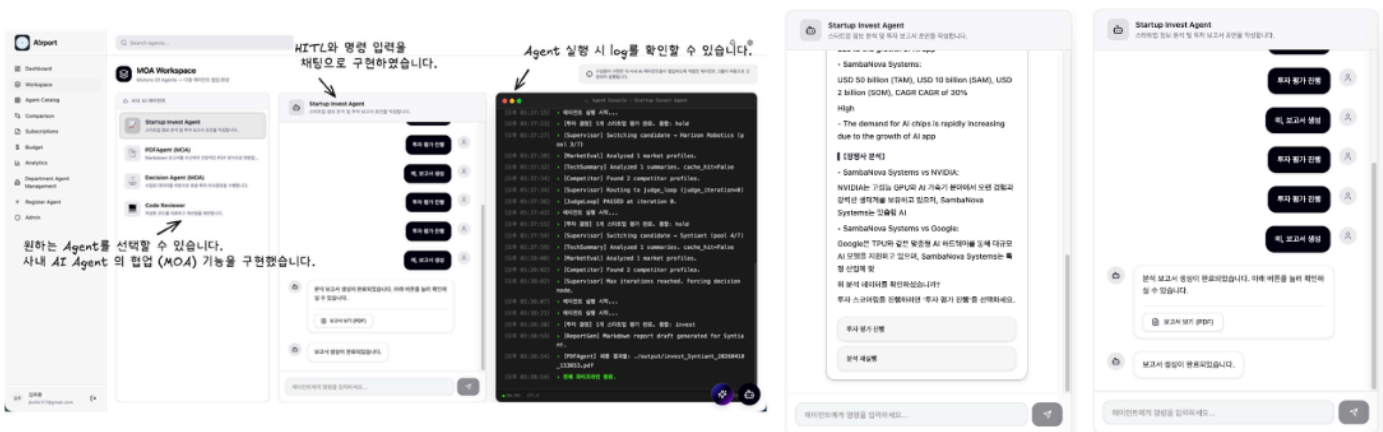
> 현재 구독 중인 에이전트 목록과 상태를 확인하는 화면이다. 에이전트명, 활성 상태, 호출당 비용, 구독 일자를 일괄 조회할 수 있으며, 불필요한 구독은 즉시 해지할 수 있다. 부서 단위로 사용 중인 에이전트를 명확히 파악하고 관리할 수 있는 구독 현황 페이지로, 비용 통제 체계의 기반이 되는 화면이다.

6. Agent Comparison



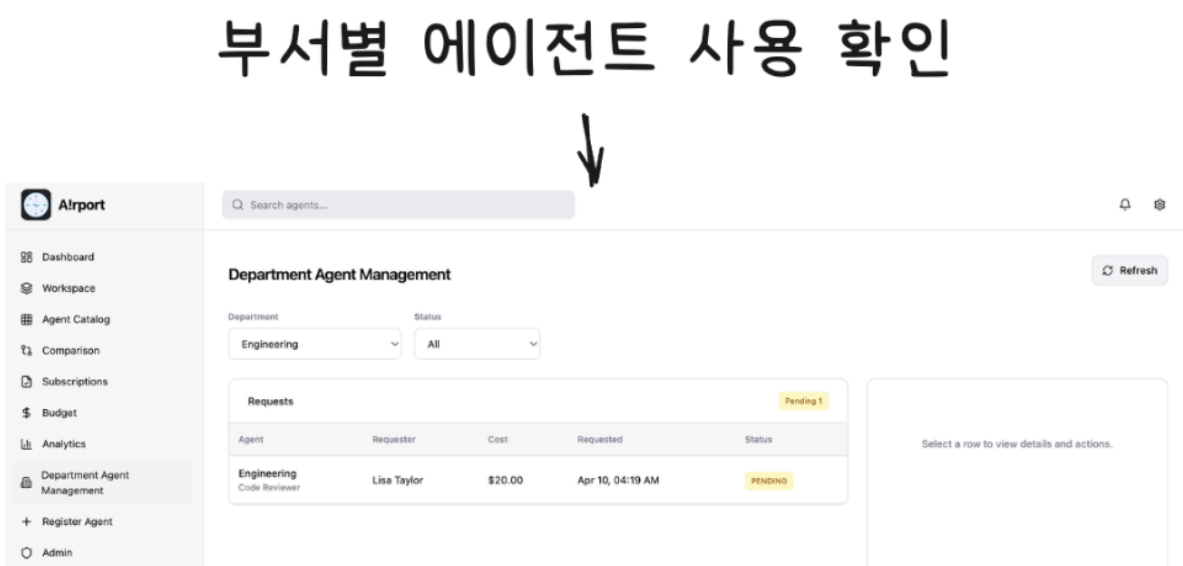
> 여러 에이전트를 동시에 선택하여 성능을 비교할 수 있는 화면이다. 레이더 차트를 통해 Performance, Cost Efficiency, Rating, Success Rate, Speed 5개 지표를 시각적으로 비교할 수 있으며, 하단 테이블에서 응답 속도, 성공률 등 정량 지표를 에이전트별로 대조 확인할 수 있다. 감각에 의존하던 에이전트 선택을 데이터 기반 의사결정으로 전환하는 핵심 화면이다.

7. MOA Workspace



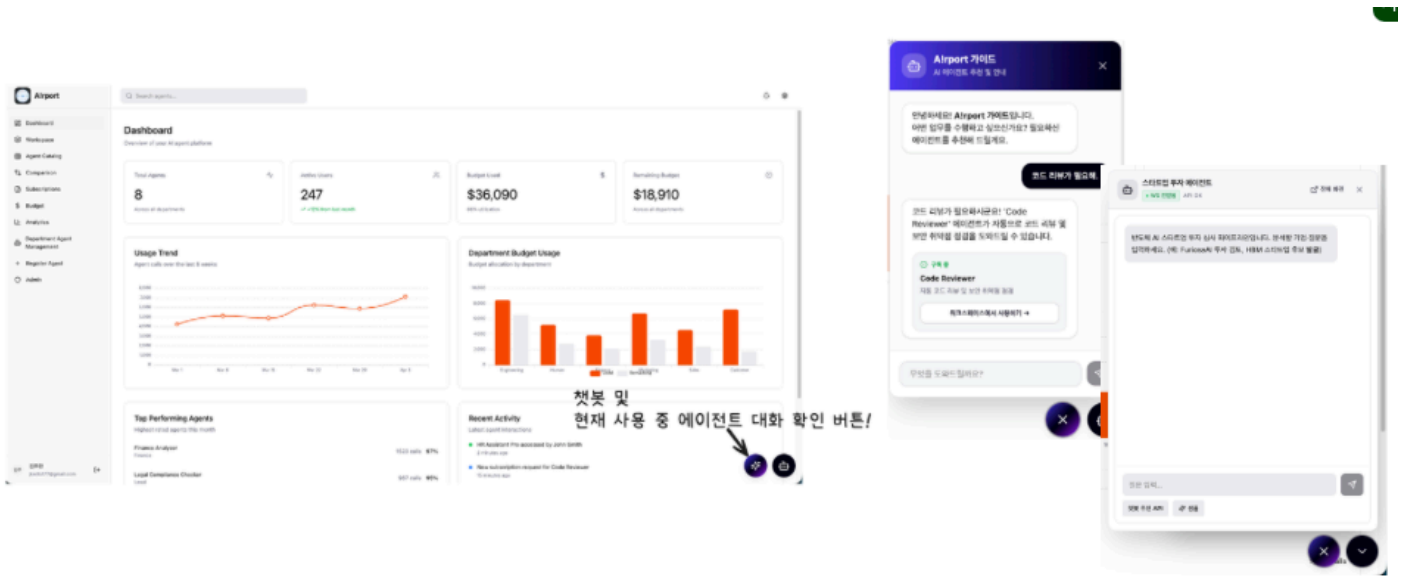
> 여러 에이전트를 조합하여 복합 업무를 수행하는 협업 실행 화면이다. 사용자는 Startup Invest Agent, PDFAgent, Decision Agent 등 목적에 맞는 에이전트를 직접 선택하여 조합할 수 있으며, 채팅 인터페이스를 통해 자연어 명령으로 에이전트를 실행한다. HITL(Human-in-the-Loop) 방식을 적용하여 사용자가 실행 흐름에 직접 개입할 수 있도록 구현하였다. 우측 콘솔에서는 에이전트 실행 과정의 로그를 실시간으로 확인할 수 있어 처리 흐름의 투명성을 확보하였다. 단일 에이전트로는 처리하기 어려운 분석·판단·생성이 결합된 복합 업무를 자동화하는 MOA 구조의 핵심 실행 화면이다.

8. Department Agent Management



> 부서 관리자가 소속 부서의 에이전트 구독 요청을 검토하고 승인·반려하는 화면이다. 부서 및 상태 기준으로 요청을 필터링할 수 있으며, 신청자, 비용, 요청 일시, 처리 상태를 한 화면에서 확인할 수 있다. 개인이 무분별하게 에이전트를 사용하는 것을 방지하고, 부서 단위의 AI 사용 권한을 체계적으로 관리하기 위한 거버넌스 화면이다.

9. 챗봇 (Airport 가이드)



> 대시보드 어느 화면에서든 접근 가능한 플로팅 챗봇 인터페이스다. 사용자가 업무 목적을 자연어로 입력하면 현재 구독 중인 에이전트 중 가장 적합한 에이전트를 자동으로 추천하며, 해당 에이전트로 즉시 연결할 수 있는 바로가기를 함께 제공한다. 별도 탐색 없이 업무 맥락에 맞는 에이전트를 즉시 찾아주는 구조로, 카탈로그 탐색과 실행 사이의 마찰을 최소화하였다. 우측 패널에서는 현재 사용 중인 에이전트와의 대화 이력도 함께 확인할 수 있다.

---

6. 결론

본 프로젝트는 조직 내 AI 에이전트 활용이 확산되는 과정에서 발생하는 관리 공백에 주목하였다. 에이전트의 존재 파악 어려움, 성능 검증 기준의 부재, 비용 통제 체계의 미비라는 세 가지 문제를 정의하고, 이를 해결하기 위한 통합 플랫폼을 설계·구현하는 것을 목표로 하였다.

MVP 범위 내에서 에이전트 카탈로그, 등록 및 구독, 비용 관리, 실행 연동, 챗봇, 성과 관리의 6개 핵심 서비스를 구현하였다. 이를 통해 개인 단위로 분산되어 있던 에이전트가 조직 자산으로 축적될 수 있는 구조를 마련하였고, AI 사용이 예산 범위 내에서 통제되며 업무 맥락에 맞게 연결될 수 있는 운영 체계를 갖추었다.

시스템은 MSA 기반으로 설계되어 각 서비스가 독립적으로 운영되며, 에이전트 성능 비교, MOA 기반 협업 자동화, KPI 연계 고도화는 후속 확장 과제로 정의하였다.

궁극적으로 본 서비스는 AI를 도입하는 것과 AI를 운영하는 것이 서로 다른 문제임을 전제로 한다. 도입 단계를 넘어 AI가 조직의 의사결정과 업무 프로세스에 구조적으로 통합되기 위해서는 자산 관리, 성능 표준화, 비용 통제, 협업 자동화를 아우르는 운영 체계가 필요하다. 본 프로젝트는 그 체계를 설계하고 검증하는 첫 단계로서 의미를 가진다.